

Security — How We Keep Data Safe

Last updated: 2026-08-29 · **Audience:** developers, IT, leadership · **Companion doc:**

`docs/DEEP_AUDIT_REPORT.md` (full technical audit)

This document explains, in plain terms, what protects employee and company data in Project-Ticket today, and what was just hardened. It reflects the state of the code as of this date — if you change auth, roles, or the API surface, update this file too.

1. The short version

Every piece of data in this app — complaints, HR records, assets, tasks — lives in one place (Firestore) and is reachable through exactly one door: our Express API. The browser never talks to the database directly. That single door is where every protection lives: who you are, what you're allowed to do, and what you're allowed to see.

Nothing here requires trusting the frontend. Every check that matters — "is this really you," "are you allowed to do this," "do you own this record" — is enforced again on the server, even though the UI also hides buttons a role shouldn't see.

2. Identity — how we know who you are

- **Login** goes through Firebase Authentication, not our own password store. We never see or hold a raw password — Firebase verifies it and hands back proof of identity.
(`main/backend/controllers/authController.js`)
- **Sessions** are represented by a signed token (JWT) issued by our server, valid for 7 days, naming the user's id, role, and a session id.
- **Logging out — or being logged out — actually works.** Each login creates a session record in the database; logout (by the user, or a forced logout by an admin) flips that record to "revoked." Every request re-checks this, so a stolen or old token stops working within about a minute of being revoked, not in 7 days.
- **Locked out after repeated wrong passwords.** Five failed attempts locks the account; only the Founder (the platform's top-level admin account, called `superadmin` in the code) can unlock it. Every failed attempt is logged with the IP it came from.
- **Rate-limited login.** On top of the lockout, no more than 10 login/register/re-verify attempts are accepted from one source every 15 minutes — slows down anyone trying to guess passwords by machine.

- **Passwords must be at least 10 characters** — enforced the same way whether you're signing up yourself, an admin is creating your account, or an admin is resetting your password. (*as of this update — previously self-registration had no real minimum*)
 - **You can't grant yourself a role.** Signing up always creates a plain employee account. Only the Founder can create hr/it/coordinator/employee accounts, through a screen that only offers those specific roles — there's no path from the signup form to picking your own permissions. Promoting an existing account to Founder-level access is a separate, even more restricted action.
-

3. Permissions — what you're allowed to do

Three layers, all checked on the server (not just hidden in the UI):

1. **Route-level role check** — e.g. only `hr` and `founder` accounts can even reach the "view all HR complaints" endpoint.
2. **Fine-grained permission matrix** — for sensitive actions like editing IT assets, a Founder-configurable table decides who can create/edit/delete, beyond the basic role check.
3. **Ownership check** — for your own records (your complaint, your leave request), you can only edit or delete the ones that are actually yours. Editing someone else's is rejected by the server even if you knew its ID.

Example, concretely: if you try to delete a complaint that isn't yours, the request is refused with a permission error — the server checks the record's owner before allowing the delete, every time, regardless of what the app's UI shows you.

4. What's protecting the API itself

PROTECTION	WHAT IT DOES
Security headers (helmet) <i>(added in this update)</i>	Every response now carries a Content-Security-Policy, HSTS, and clickjacking/MIME-sniffing protections by default — closes off a whole class of browser-side attacks.
General rate limiting <i>(added in this update)</i>	Every API route (not just login) is now capped at 300 requests per 15 minutes per source, in addition to the stricter login limiter — stops runaway scripts or a compromised account from hammering the database.
Trusting the real client IP <i>(fixed in this update)</i>	We deploy behind Vercel's proxy. The server now correctly reads the real visitor IP from the proxy instead of seeing one shared address for everyone — this is what makes the rate limits above actually apply per-visitor instead of accidentally lumping all traffic together.
CORS allow-list	Only our own frontend's web address (plus localhost, for development) may call this API from a browser — not "anyone on the internet."
Generic error messages	If something breaks unexpectedly, the client sees "Internal server error" — never a raw database error, stack trace, or internal file path. The real detail is only ever written to the server's own log.
File upload limits	Uploaded HR documents are checked by file type (PDF/JPG/DOC/DOCX only) and capped at 10MB before they're ever written to storage.
Outbound email is escaped	Complaint text going into notification emails is sanitized first, so a complaint can't be crafted to inject links or formatting into HR's inbox.

5. Where data actually lives

- **Firestore** (Google's database) holds everything: user accounts, complaints, leave requests, assets, tasks, audit logs. Nothing is duplicated into a second database.
 - **Firestore Storage** holds uploaded HR documents. Files aren't publicly reachable by guessing a URL — every download goes back through our API's own permission checks.
 - **Only our server has the keys.** The credentials that let anything talk to Firestore/Storage/Auth live only in the backend's environment variables, never in the frontend bundle a browser downloads, and are excluded from git.
 - Session tokens live in the browser's local storage, the standard place for this kind of app. This means, like most apps built this way, a serious enough bug that let an attacker run their own code on our pages could expose someone's session — which is exactly why the security headers above matter: they're a big part of what prevents that from being possible in the first place.
-

6. What we log

- **Every admin action is recorded:** creating/editing/deleting a user account, changing someone's permissions, resetting a password. If something goes wrong, there's a trail of who did what and when.
 - **Every failed login is recorded**, with the source IP, feeding the lockout mechanism above.
 - Regular request/error activity currently goes to the server's own console log rather than a separate searchable log system — good enough for the app's current size, worth revisiting as usage grows (see §7).
-

7. Known trade-offs (nothing hidden)

We'd rather tell you what's still a work-in-progress than pretend everything is perfect:

- **No multi-factor authentication yet.** Account security today rests on password strength plus the lockout/rate-limit protections above.
- **No dedicated searchable logging system yet** — day-to-day request logs live in the server's console output, not a queryable dashboard. Admin actions and failed logins are the exception (those go to the database, see §6).
- **No automated pipeline** runs security/dependency checks before a deploy yet — deploys go out via Vercel directly from the codebase.
- A couple of internal endpoints (production-render updates, and an employee updating their *own* task's status) are intentionally open to any logged-in account rather than role-restricted — reviewed and confirmed low-risk given what they touch, but worth re-checking if the app's scope grows.

None of these block using the app today; they're the honest next-hardening list, tracked in

`docs/DEEP_AUDIT_REPORT.md` .

8. If something looks wrong

If you notice unexpected access, a locked-out account that shouldn't be, or anything that looks like it shouldn't have been possible — the Founder can see and revoke active sessions, view failed-login history, and force-unlock accounts from the Security section of the Founder dashboard (`securityRoutes.js` , Founder-only).